

# Zephyr Driver Development

Zephyr RTOS Course

The logo for Iomico, featuring the word "iomico" in a lowercase, sans-serif font. The letters are colored: 'i' is yellow, 'o' is red, 'm' is green, 'i' is blue, 'c' is green, and 'o' is blue. The logo is set against a white, rounded rectangular background.



## Quick Recap



- Custom board creation: **Copy & Rename** vs. **From Scratch**
- Board directory structure and required files
- Kconfig & DTS roles in board definition
- Building with `west build -b our_board`



## Format



- Lecture with optional **live demo**
- Homework after each session (checked by us)
- Recordings will be shared
- Questions addressed at the end in a **Q&A session**



## Today's Agenda



- Driver fundamentals in Zephyr
- Module vs driver (what is what)
- Minimal module layout and required files
- DTS binding and compatible linkage
- Kconfig and CMake integration
- Driver implementation pattern (Sensor API)
- Device instantiation and multi-instance behavior
- Homework



# Drivers in Zephyr





## What is a Zephyr Driver?



- A **software component** that abstracts a hardware peripheral
- Implements a **standard Zephyr API** (Sensor, GPIO, UART, SPI, etc.)
- Usually controlled by **Kconfig** and instantiated from **Device Tree**
- Lets applications stay **hardware-agnostic** across boards



## Module vs. Driver



- A **module** is a *packaging concept* — a West-managed repository
- A **driver** is a *code pattern* — implements a hardware abstraction

|          | Module                              | Driver                                        |
|----------|-------------------------------------|-----------------------------------------------|
| What     | External repo managed by West       | C code following Zephyr's device driver model |
| Contains | Drivers, libraries, HALs, utilities | API implementation for specific hardware      |
| Examples | mbedTLS, littlefs, vendor's HALs    | sensors, UART, SPI, GPIO drivers, etc.        |

In this lecture, a module contains a driver. Not all modules are drivers.



## Driver Lifecycle



- DTS node declares `compatible = "our,driver"`
- Binding defines that compatible string
- Kconfig symbol enables compilation
- CMake compiles driver sources
- Driver instance is created from DTS
- Application gets device handle and uses API





# Repository Layout



## Layout



```
our-driver-module/
├── include/
│   └── our_drivers/
│       └── our_driver.h      # Public C API definition
├── drivers/
│   └── our_driver/
│       ├── our_driver.c     # Driver implementation
│       ├── CMakeLists.txt   # Build logic
│       └── Kconfig          # Config menu
├── dts/
│   ├── bindings/
│   └── driver/
│       └── our,driver.yaml  # Device Tree binding
├── CMakeLists.txt           # Module root build directives
└── Kconfig                  # Module root config menu
```



# Driver Module Files Architecture



## Module Registration & Build

zephyr/module.yml

CMakeLists.txt  
(root)

Kconfig  
(root)

West discovery &  
module-level config

## Device Tree Bindings

our\_driver.yaml  
(binding)

Defines compatible  
string & properties

*Links DT → Kconfig  
auto-generation*

## Driver Implementation

our\_driver.c

our\_driver.h  
(public API)

CMakeLists.txt  
(driver)

Kconfig  
(driver)

Driver logic &  
configuration



module.yml





zephyr/module.yml



```
name: our_custom_drivers
build:
  cmake: .
  kconfig: Kconfig
  settings:
    dts_root: .
```

- Registers module with West
- Tells Zephyr where to find CMake, Kconfig, and DTS bindings



# DTS Binding and Compatible



## Device Tree Node



```
/ {  
    sensor: temp_sensor {  
        compatible = "our,driver";  
        reg = <0x76>;  
        calibration = <100>;  
        status = "okay";  
    };  
};
```

The `compatible` value is the key that links DTS to this driver.



Binding File: `our,driver.yaml`



```
description: Our custom sensor driver binding
compatible: "our,driver"
include: i2c-device.yaml

properties:
  calibration:
    type: int
    required: true
```

- Defines schema for DTS nodes that use this compatible
- Enables auto-generated `DT_HAS_OUR_DRIVER_ENABLED`





## Fetching DTS Properties: `DT_INST_PROP`



```
/* In the instantiation macro – inst comes from DT_INST_FOREACH_STATUS_OKAY */  
static const struct our_driver_config cfg_##inst = {  
    .i2c          = I2C_DT_SPEC_INST_GET(inst),  
    .calibration = DT_INST_PROP(inst, calibration),  
};
```

- `DT_INST_PROP(inst, calibration)` resolves to the integer in the DTS node
- Requires `#define DT_DRV_COMPAT our_driver` at the top of the file
- The property name must match the binding **exactly** (underscores for hyphens)
- Result is a **compile-time constant** – zero runtime overhead





```
#define DT_DRV_COMPAT our_driver
```

- DTS uses comma form: `our,driver`
- C macro uses underscore form: `our_driver`
- This enables instance macros like `DT_INST_FOREACH_STATUS_OKAY(...)`



## DTS to Driver Connection



### 1) Device Tree Node

`compatible = "our,driver"`



### 2) Driver Match

```
#define DT_DRV_COMPAT  
our_driver
```



### 3) App Gets Device

```
DEVICE_DT_GET_ANY  
(our_driver)
```



### 4) Runtime API Usage

```
device_is_ready(dev)  
sensor_sample_fetch(dev)
```



# Kconfig and CMake



## Driver Kconfig Menu



- Located at `drivers/our_driver/Kconfig`.
- Auto-enables the driver when the DT node is present.

```
menuconfig OUR_DRIVER
    bool "Our Custom Driver"
    default y
    depends on DT_HAS_OUR_DRIVER_ENABLED # Only if HW is present in the DT
    select I2C                          # Auto-pulls dependencies
    help
        Enable the custom driver for the I2C peripheral
```





- Located at `drivers/our_driver/CMakeLists.txt`.

```
if(CONFIG_OUR_DRIVER)
    zephyr_library()
    zephyr_library_sources(our_driver.c)
endif()
```

### Flow:

1. DTS declares node with `compatible = "our,driver"`
2. Kconfig is enabled `DT_HAS_OUR_DRIVER_ENABLED=y`
3. Kconfig defaults `OUR_DRIVER=y`
4. `our_driver.c` is compiled



# Driver Implementation



## Rules & Guidelines



- Zephyr drivers are written in **C** (not C++)
- Drivers following standard APIs (e.g., `sensor`) inherit **Zephyr infrastructure**
  - `sensor_sample_fetch()` / `sensor_channel_get()`
- Each driver file starts with:

```
#define DT_DRV_COMPAT our_driver
```

This links the C code to every DT node with `compatible = "our,driver"`.



## Multiple Driver Models



- **Sensor API** `zephyr/drivers/sensor.h`
- **GPIO** `zephyr/drivers/gpio.h`
- **PWM** / `zephyr/drivers/pwm.h`
- **I2C** `zephyr/drivers/i2c.h`
- **SPI** `zephyr/drivers/spi.h`
- **UART** `zephyr/drivers/uart.h`
- If no standard class fits, provide a **custom API**





```
#define OUR_DRIVER_DEFINE(inst) \
    static struct our_driver_data data_##inst; \
    static const struct our_driver_config cfg_##inst = { \
        .i2c = I2C_DT_SPEC_INST_GET(inst), \
    }; \
    DEVICE_DT_INST_DEFINE(inst, \
        our_driver_init, NULL, \
        &data_##inst, &cfg_##inst, \
        POST_KERNEL, CONFIG_SENSOR_INIT_PRIORITY, \
        &our_driver_api)

DT_INST_FOREACH_STATUS_OKAY(OUR_DRIVER_DEFINE)
```





## Init Levels



| Level        | When                       | Typical Use                                                   |
|--------------|----------------------------|---------------------------------------------------------------|
| EARLY        | Before any kernel services | Architecture-specific HW (e.g., interrupt controller)         |
| PRE_KERNEL_1 | Before kernel memory mgmt  | Basic HW: clocks, pin mux, GPIO controllers                   |
| PRE_KERNEL_2 | After PRE_KERNEL_1         | HW depending on PRE_KERNEL_1 drivers (e.g., SPI bus)          |
| POST_KERNEL  | Kernel is running          | Most drivers — <b>sensors, I2C devices, UART</b>              |
| APPLICATION  | After POST_KERNEL          | App-level setup, high-level subsystems                        |
| SMP          | After secondary CPUs start | Per-CPU init on multi-core SoCs (requires <b>CONFIG_SMP</b> ) |



# App Integration Demo



## Use Device From Application



```
const struct device *dev = DEVICE_DT_GET_ANY(our_driver);

if (!device_is_ready(dev)) {
    return -ENODEV;
}

sensor_sample_fetch(dev);
```

- App remains decoupled from board pin mappings and bus details



## End-to-End Checklist



- DTS node added with `compatible = "our,driver"`
- Binding exists and is discoverable
- Kconfig symbol visible and enabled
- Driver source compiled by CMake
- Device is ready at runtime
- API calls return expected data





Homework



## Home Task 1



- Create a sensor driver following the structure shown in this lecture.
- The sensor will be a simple on-board led.
- The driver should implement `sensor_sample_fetch` and `sensor_channel_get`, turning on/off the led, respectively.
- Commit and tag it **l6-task1**.



## Home Task 2



- Add a custom extension API function to your driver
- Call it from `main.c`.
- This function will change a parameter of your choosing in the dynamic data struct.
- Commit and tag it **l6-task2**.

# Thank You

Questions?







1. [Zephyr Modules \(Docs\)](#)
2. [Zephyr Sensor API Reference](#)
3. [Device Tree Bindings](#)
4. [Device Driver Model](#)